



als lokales Datenbanksystem

Klaus Prinz | Datenbanken - BIM25 - WIT

Agenda

Ein kompakter Überblick von architektonischen Grundlagen zur praktischen Anwendung in Python.

- **Grundlagen & Konzepte:** Was ist SQLite, welche Ideen drückt es aus, wofür ist es (nicht) geeignet, wie unterscheidet es sich von Flat Files und Client-Server-DBs?
- **SQLite und Python:** Wie kann man SQLite mit Python verwenden, welche Vorteile bietet die Verbindung mit Pandas für die Datenanalyse, was sind ORMs und wie nutzt man sie?
- **Live-Demo:** Hands-On in einem Jupyter Notebook.

Wesentliches

SQLite ist das am häufigsten installierte Datenbanksystem der Welt – unsichtbar integriert in Smartphones, Browser und Betriebssysteme.

Zentrale Eigenschaften:

- **Serverless:** Kein separater Datenbankserver (wie PostgreSQL oder MySQL) notwendig.
- **Self-Contained:** Gesamte Datenbank besteht aus genau **einer einzigen Datei**.
- **Zero-Configuration:** Kein Setup, kein "Installieren und Warten", läuft sofort "Out-of-the-Box".
- **Public Domain:** Quellcode ist frei von Urheberrechtsansprüchen und für jeden Zweck frei nutzbar.

Entwicklung und Idee

Ein relationales Datenbanksystem, das ohne Datenbankadministrator (DBA) und ohne administrative Hürden lokal betrieben werden kann.

- **Entwicklung:** Im Jahr 2000 von D. Richard Hipp (ursprünglich für US-Kriegsschiffe zur Fehlertoleranz bei Systemausfällen).
- **Kernidee:** Statt eines großen, zentralen Servers, mit dem man über das Netzwerk kommuniziert, wandert die gesamte Datenbank als **selbstständige Datei direkt auf die Festplatte**.

Datenbehauungen

Flat Files vs. SQLite

Man skriptet schnell etwas, speichert Daten einfach in JSON- oder CSV-Dateien - und irgendwann wird es unübersichtlich.

CSV / JSON / Excel	SQLite (Datenbank)
Laden ganzer Dateien in den Arbeitsspeicher	Gezielte Abfragen dank Indizierung (<code>SELECT</code> mit <code>WHERE</code>)
Keine Datentyp-Integrität (Dateien korrumpieren leicht)	ACID-konform (Transaktionssicherheit & definierte Typen)
Keine Mechanismen bei gleichzeitigen Schreibvorgängen	Atomares File-Locking (Concurrency Control) verhindert Datenverlust

Einsatzgebiete und Eignungen

SQLite vs. Client-Server-DBs (PostgreSQL / MySQL)

Ist SQLite jetzt die Lösung für alles, oder?

SQLite (Lokal & Serverless)	Client-Server-DB (PostgreSQL / MySQL)
Lokale Prototypen, Skripte & Jupyter Notebooks	Multi-User Webanwendungen mit hoher Last
Geringe Komplexität (keine Server-Administration)	Feingranulare Benutzerrechte (ACLs pro Zeile/Tabelle)
Einzelne Datei auf der Festplatte / lokal	Zentraler Datenbank-Server über Netzwerk
Grenze: Hohe gleichzeitige Schreiblast (Write-Locking)	Skalierbar bei parallelen Schreibvorgängen

SQLite in Python

Der integrierte Standardweg

Python bringt alles mit, was man braucht: Das Modul `sqlite3` ist standardmäßig in der Python-Standardbibliothek enthalten.

- **Keine externen Treiber** oder pip-Installationen für den Einstieg nötig.
- **Typischer Ablauf:** Verbindung herstellen (`connect`), Cursor erzeugen, SQL-Befehl ausführen, Änderungen bestätigen (`commit`), Verbindung schließen.

Kurzer Blick unter die Haube

Code-Beispiel: Verbindung & Abfrage

```
import sqlite3

# Verbindung herstellen (erstellt die Datei, falls sie nicht existiert)
conn = sqlite3.connect("data/library.db")
cursor = conn.cursor()

# SQL-Abfrage ausführen
cursor.execute("SELECT title, author FROM books WHERE year > 2020;")

# Ergebnisse abrufen und ausgeben
for row in cursor.fetchall():
    print(f"Titel: {row[0]}, Autor: {row[1]}")

conn.close()
```


Brücke zur Datenanalyse

SQLite + Pandas = Jupyter Playground

Daten direkt aus einem Pandas DataFrame in die Datenbank schreiben und wieder einlesen – mit zwei Zeilen Code.

```
import pandas as pd
import sqlite3

conn = sqlite3.connect("data/library.db")

# DataFrame in SQLite-Tabelle speichern
df.to_sql("books", conn, if_exists="replace", index=False)

# SQL-Abfrage direkt als DataFrame einlesen
df_result = pd.read_sql("SELECT * FROM books WHERE year > 2020", conn)
```

- **Ideal für Datenanalyse:** Große Metadatensätze (z. B. MARC-Records oder Katalog-Exports als CSV) einlesen, lokal bereinigen, in SQLite persistieren und interaktiv in Jupyter Notebooks analysieren.

Ein wenig ORM: SQLAlchemy

Vom rohen SQL zur objektorientierten Welt

Wer sind diese Object-Relational Mapper (ORMs) und warum sollte man sie nutzen?

- **Das Prinzip:** Datenbank-Logik wird in Python-Logik übersetzt. Tabellen werden als Python-Klassen abgebildet; Zeilen entsprechen konkreten Objekten.
- **Vorteile:**
 - Agnostisches Interface für verschiedene Datenbank-Anbieter.
 - Unabhängigkeit vom konkreten SQL-Dialekt.
 - Typsicherheit und automatisiertes Mapping.
- **Beispiel:** Statt `cursor.execute("SELECT ...")` arbeitet man direkt mit logischen Objekten wie `session.query(Book).filter(...)`.

Nächster Schritt: Live-Demo!

Hands-On in einem Jupyter Notebook

Praxisumgebung: Start in VS Code mit Extensions: Python, Jupyter, SQLite Viewer.

- Inhalt der Demo:
 - i. Tabellen-Setup & Import von Beispieldaten
 - ii. SQL-Abfragen & Joins (Patrons, Books, Loans)
 - iii. Kurzer Einblick: ORM mit SQLAlchemy

Quellen & Weiterführende Links

Offizielle Dokumentationen

- SQLite: (sqlite.org).
- Python `sqlite3` -Modul: (docs.python.org).
- Pandas: (pandas.pydata.org).
- SQLAlchemy: (sqlalchemy.org).

Datenanalyse-Kurs

- freeCodeCamp: (freecodecamp.org/learn/data-analysis-with-python).

Vielen Dank für Ihre Aufmerksamkeit!

Fragen & Diskussion

Materialien & Code-Beispiele:

 <https://github.com/klausbprinz/sqlite-demo>